

Cleft — สรุปโปรเจกต์

1. นิยามปัญหา (Problem Definition)

ปัญหาที่โปรเจกต์นี้พยายามแก้ไขคือความยุ่งยากในการจัดการค่าใช้จ่ายเมื่อกลุ่มคนไปทานอาหาร หรือออกไปทำกิจกรรมร่วมกัน แต่ละคนอาจจ่ายเงินคนละแบบ ไม่ได้ระบุชัดว่าใครจ่ายอะไร อะไรเป็นค่าใช้จ่ายของใคร จึงทำให้เกิดปัญหาดังต่อไปนี้:

- คนในกลุ่มไม่ทราบว่าต้องจ่ายเท่าไรต่อคน
- มีการคำนวณแยกกันเองและเกิดความผิดพลาดได้ง่าย
- ข้อมูลค่าใช้จ่ายกระจัดกระจาย เช่น บันทึกในแชต ไฟล์ Excel หรือการจดด้วยมือ
- ไม่มีกลไกตรวจสอบว่าคนที่จ่ายแล้ว ได้รับคืนจริงหรือไม่
- ความยุ่งยากเพิ่มขึ้นเมื่อมีรายการที่หลายคนร่วมใช้กัน หรือมีคนใช้บางรายการแต่ไม่ใช่ทุกรายการ

Cleft จึงถูกออกแบบมาเพื่อช่วยให้คนในกลุ่มสามารถ:

- เพิ่มบิล/ปาร์ตี้ใหม่
- ใส่รายการค่าใช้จ่ายแต่ละรายการ
- ระบุผู้เข้าร่วมแต่ละคนในปาร์ตี้
- กำหนดรูปแบบการแบ่งเงิน เช่น แบ่งให้ทุกคน หรือแบ่งตามรายการที่เลือก
- ตรวจสอบยอดสรุปก่อนยืนยัน
- บันทึกประวัติและข้อมูลสรุปลงในฐานข้อมูลเพื่อให้สามารถดูย้อนหลังได้

2. Scope ของโปรเจกต์

2.1 Scope ที่อยู่ในงาน

- ระบบล็อกอินด้วย Google OAuth
- การจัดการผู้ใช้และข้อมูล profile
- การสร้าง party/บิลสำหรับกลุ่ม
- การเพิ่มรายการค่าใช้จ่าย (Item) พร้อมบันทึกราคาและหมายเหตุ
- การเพิ่มสมาชิกใน party
- การแบ่งค่าใช้จ่ายแบบ ALL และ PARTIAL
- การคำนวณยอดที่แต่ละคนต้องจ่าย/ได้รับคืน
- การยืนยันบิล (Confirm) เพื่อบันทึกข้อมูลแบบถาวร
- การเก็บประวัติการทำธุรกรรมหรือ party ที่เคย confirm แล้ว
- การจัดเก็บข้อมูลใน PostgreSQL ผ่าน Prisma ORM
- การใช้งานแบบ local draft session ก่อนยืนยันข้อมูลสุดท้าย

2.2 Scope ที่อยู่นอกขอบเขต

- การชำระเงินจริงผ่าน e-wallet หรือธนาคาร
- ระบบแจ้งเตือนอัตโนมัติ เช่น SMS, Line, Email
- การทำงานหลายเครื่องแบบ real-time แบบ live sync
- การคำนวณสกุลเงินหลายสกุลและอัตราแลกเปลี่ยน
- รายงานสถิติขั้นสูงหรือ analytics แบบเชิงธุรกิจ
- การจัดการ recurring bill หรือ subscription

สิ่งที่โปรเจกต์นี้เน้นคือ “ระบบจัดการแบ่งบิลสาธารณะในกลุ่ม” ให้ทำงานได้จริงและมีโครงสร้างที่เข้าใจง่าย ไม่ได้ถูกออกแบบให้เหมือนแอปชำระเงินแบบเต็มรูปแบบ

3. เหตุผลที่ใช้โครงสร้างโปรเจกต์แบบนี้

โครงสร้างของโปรเจกต์นี้ถูกออกแบบให้แยกความรับผิดชอบแต่ละชั้นออกจากกัน เพื่อให้ทีมพัฒนาเข้าใจง่ายและจัดการโค้ดได้สะดวกขึ้น

3.1 ชั้น Frontend

- โฟลเดอร์: frontend/
- ใช้ Next.js
- ทำหน้าที่รับข้อมูลจากผู้ใช้ แสดงหน้า UI และจัดการสถานะหน้า เช่น dashboard, items, members, summary
- มีฟังก์ชันควบคุม local draft session และเรียก API ไปที่ backend
- เหตุผล: Frontend ต้องแยกออกจาก backend เพื่อให้มีหน้าที่ชัดเจนและสามารถ deploy โดยแยก service ได้

3.2 ชั้น Backend

- โฟลเดอร์: backend/
- ใช้ NestJS + Prisma
- ทำหน้าที่รับ request, ตรวจสอบ auth, validate input, คำนวณค่าใช้จ่าย, และจัดเก็บลงฐานข้อมูล
- เหตุผล: Backend เป็นส่วนที่ควบคุมข้อมูลและ business logic เพื่อให้ data ถูกบันทึกในรูปแบบที่สอดคล้องและปลอดภัย

3.3 ชั้น Database

- ใช้ PostgreSQL
- เก็บข้อมูลต่าง ๆ เช่น User, Party, Item, Participant, Consumption, History
- เหตุผล: ข้อมูลที่เกี่ยวกับค่าใช้จ่ายต้องถูกเก็บถาวรและสามารถเรียกใช้ในภายหลังได้

3.4 ชั้น Dev Helper / Tooling

- scripts/: เก็บสคริปต์ช่วยงาน เช่น สร้าง JWT secret และแปลง Markdown เป็น PDF
- edit_front_end/: เป็นพื้นที่ทดลอง UI หรือเวอร์ชัน mockup ที่นำมาอ้างอิงสำหรับการออกแบบ
- uploads/: ใช้จัดเก็บไฟล์แนบ เช่น slip รูปสลิปค่าใช้จ่าย

3.5 Root-level configuration

- Dockerfile: สำหรับ build backend ใน container
- docker-compose.yml: สำหรับรันหลาย service พร้อมกันในเครื่อง local
- .env.example: template variable ที่ใช้สำหรับ local/dev

โครงสร้างนี้ทำให้แต่ละชั้นมีความรับผิดชอบที่ชัดเจน เช่น

- UI ไม่น่าจะมี business logic มากเกินไป
- API ไม่ต้องคอยจัดการ rendering
- Database ทำหน้าที่เก็บข้อมูลอย่างเดียว

4. Data Pipeline (การไหลของข้อมูล)

การทำงานของระบบสามารถอธิบายเป็น pipeline ได้ดังนี้:

```

ผู้ใช้ทำงานบน Frontend
↓
Next.js รับข้อมูลและจัดเก็บ draft ใน localStorage
↓
Frontend ส่ง request ไปที่ backend ผ่าน /api
↓
NestJS ตรวจสอบ auth, validate input, คำนวณ split
↓
Prisma ORM บันทึกข้อมูลลง PostgreSQL
↓
Frontend ดึงข้อมูลที่บันทึกแล้วมาแสดงผล
  
```

4.1 ขั้นตอนการไหลของข้อมูลแบบจริง

1. ผู้ใช้เข้ามาที่หน้า app และทำการล็อกอินด้วย Google
2. ระบบทำการสร้าง session และเก็บข้อมูลผู้ใช้ใน backend
3. ผู้ใช้สร้าง party ใหม่และเพิ่มรายการค่าใช้จ่าย
4. ผู้ใช้เพิ่มสมาชิกของกลุ่มและเลือกการแบ่งเงิน
5. ระบบเก็บข้อมูล draft ลง localStorage ก่อนยืนยันยอดสุดท้าย
6. เมื่อกดยืนยัน ระบบจะส่งข้อมูลไปยัง backend API
7. Backend ทำการตรวจสอบว่าผู้ใช้เข้าสู่ระบบจริงหรือไม่

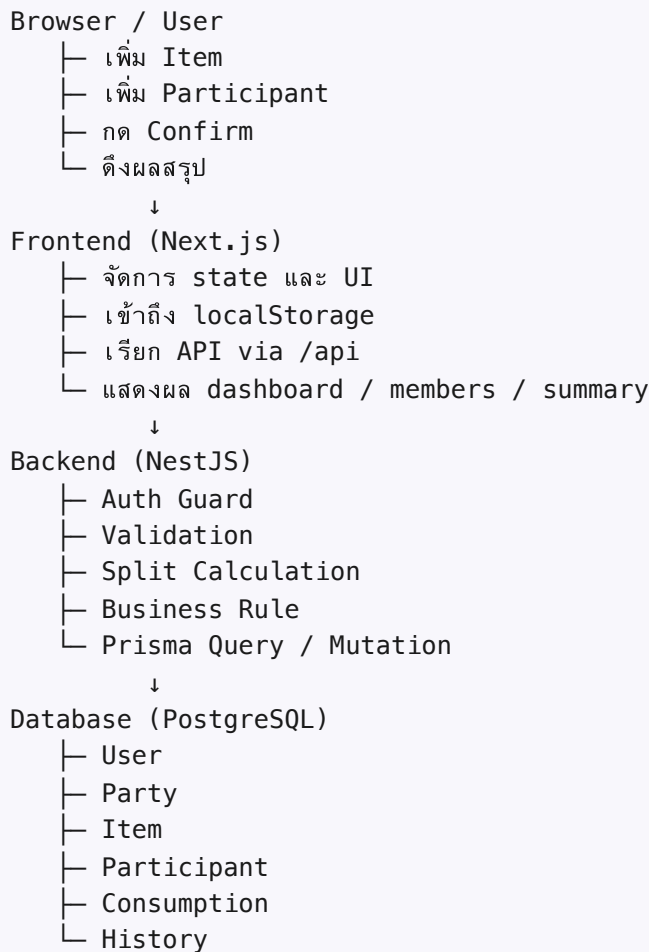
8. Backend จะประมวลผลข้อมูล, คำนวณค่าใช้จ่าย, และสร้างความสัมพันธ์ระหว่าง party, item, participant, consumption
9. Prisma บันทึกข้อมูลทั้งหมดลง PostgreSQL
10. Frontend โหลดข้อมูลใหม่จาก backend เพื่อแสดง summary, history, และรายงานสรุป

4.2 ความสำคัญของ pipeline นี้

- Frontend ทำหน้าที่แสดงและรับ input
- Backend ทำหน้าที่คุมครอง logic และความถูกต้องของข้อมูล
- Database ทำหน้าที่เก็บข้อมูลที่ผ่านประมวลผลแล้ว

จึงช่วยลดความเสี่ยงของการสร้างข้อมูลบกพร่องหรือข้อมูลไม่สอดคล้องกันระหว่าง frontend และ database

5. Data Flow (การไหลของข้อมูลเชิงระบบ)



5.1 ตัวอย่าง flow ของการคำนวณค่าใช้จ่าย

- ผู้ใช้สร้าง party ชื่อว่า "Dinner with Friends"

- เพิ่มรายการ เช่น ข้าว 300, เครื่องดื่ม 180, ของหวาน 240
- เพิ่มผู้ร่วม เช่น Alice, Bob, Charlie
- กำหนด splitType ให้กับแต่ละคน เช่น ALL หรือ PARTIAL
- ระบบจะสร้างความสัมพันธ์ระหว่าง participant กับ item ผ่าน Consumption
- จากข้อมูลดังกล่าว ระบบจะคำนวณว่า:
- ใครเป็นคนจ่ายรวม
- ใครเป็นคนต้องจ่ายเพิ่ม
- ใครควรได้รับเงินคืน
- จำนวนสรุปสุดท้ายของแต่ละคน

5.2 บทบาทของ middleware และ auth

ความปลอดภัยของระบบจะถูกควบคุมผ่าน:

- Google OAuth
- JWT strategy
- protected routes
- cookie-based auth

ดังนั้น frontend ที่ไม่มี session ถูกต้องจะไม่สามารถเข้าถึงข้อมูลส่วนตัวของ party ได้

6. ER Diagram (Diagram ความสัมพันธ์ของฐานข้อมูล)

User

- id
- googleId
- email
- username
- avatar
- currencySymbol
- refreshTokenHash
- createdAt
- updatedAt

|

| 1 —————< many

v

History

- id
- userId
- partyId
- createdAt

Party

- id
- name
- date
- totalAmount
- slipUrl
- createdAt
- updatedAt

|

| 1 —————< many

v

Item

- id
- name
- price
- note
- partyId

Party

|

| 1 —————< many

v

Participant

- id
- name
- splitType
- partyId

Item	1	————< many	Consumption	> many	———— 1	Particip
- id			- participantId			
- name			- itemId			
- price			(composite primary key)			
- partyId						

6.1 ความสัมพันธ์หลักใน ER model

- User กับ History: one-to-many
- History เชื่อมกับ Party: many-to-one
- Party กับ Item: one-to-many
- Party กับ Participant: one-to-many
- Item กับ Consumption: one-to-many
- Participant กับ Consumption: one-to-many

6.2 ความสำคัญของ Consumption

Consumption เป็นตารางที่ทำหน้าที่เชื่อมโยงระหว่าง participant กับ item เพื่ออธิบายว่า “คนคนนี้ใช้รายการนี้หรือไม่” ซึ่งเป็นประเด็นที่สำคัญมากในการคำนวณค่าใช้จ่ายและการแบ่งสัดส่วน

ถ้าไม่มีตารางนี้ ระบบจะไม่สามารถรู้ได้ว่าแต่ละคนใช้อะไรบ้าง และไม่สามารถคำนวณได้ว่าคนไหนต้องจ่ายเท่าไร

7. Business Logic หลักของแอป

หัวใจหลักของแอปคือการคำนวณค่าใช้จ่ายที่ถูกต้องและสามารถยืนยันได้

7.1 การคำนวณแบบ ALL

เมื่อ participant มี splitType = ALL หมายความว่าคนนี้จะรับผิดชอบค่าใช้จ่ายทั้งหมดของรายการนั้น ๆ

7.2 การคำนวณแบบ PARTIAL

เมื่อ participant มี splitType = PARTIAL หมายความว่าคนนี้จะรับผิดชอบเฉพาะรายการที่เลือกหรือรายการที่เป็นของตัวเองเท่านั้น

7.3 การคำนวณยอดสรุป

ระบบจะนำข้อมูลต่อไปนี้มารวมกัน:

- ราคารวมทั้งหมดของ party
- รายการแต่ละรายการและผู้ใช้จ่ายรายการนั้น ๆ
- ผู้ที่จ่ายเงินไปแล้ว

- ผู้ที่ต้องชำระเพิ่มเติมหรือได้รับคืน

จากข้อมูลนี้ จะมีการแปลงเป็นผลลัพธ์ที่สามารถอ่านได้ง่าย เช่น:

- Alice จ่าย 600 บาท
- Bob ต้องชำระ 250 บาท
- Charlie ควรได้รับคืน 150 บาท

8. เทคโนโลยีที่ใช้

ส่วน	เทคโนโลยี	หน้าที่
Frontend	Next.js	UI และ route หน้าเว็บ
Backend	NestJS	API, auth, business logic
Database	PostgreSQL	เก็บข้อมูลหลัก
ORM	Prisma	map model กับ database
Auth	Google OAuth + JWT	การยืนยันตัวตน
Container	Docker	รันแอปแบบมาตรฐาน
Local dev	docker-compose	รันหลาย service พร้อมกัน

9. Workflow ของผู้ใช้ (User Journey)

1. Login ด้วย Google
2. สร้าง party
3. เพิ่มสมาชิก
4. เพิ่มรายการค่าใช้จ่าย
5. เลือก split type
6. ตรวจสอบสรุป
7. Confirm ข้อมูล
8. ดูประวัติ party
9. ดูยอดชำระที่ต้องจ่าย/ได้รับคืน

กระบวนการนี้สะท้อนถึงแนวคิดที่ว่าแอปนี้ไม่ได้เป็นแค่ “ฟอร์มกรอกข้อมูล” แต่เป็น “ระบบควบคุมการจัดการเงินของกลุ่ม” ที่มีการยืนยันและบันทึกข้อมูลแบบมีประสิทธิภาพ

10. ประโยชน์ของโปรเจกต์

- ช่วยอำนวยความสะดวกสำหรับการแบ่งบิลแบบกลุ่ม
- ลดความผิดพลาดจากการคำนวณด้วยมือ
- เพิ่มความโปร่งใสในการจัดการค่าใช้จ่าย
- ทำให้ข้อมูลถูกจัดเก็บอย่างเป็นระบบ
- เป็นตัวอย่างโครงการ full-stack ที่แสดงความรู้ด้าน frontend, backend, database, auth, และ deployment

11. สรุปท้าย

Cleft เป็นโปรเจกต์ที่ออกแบบเพื่อแก้ปัญหการแบ่งค่าใช้จ่ายในกลุ่มคนที่ไปทานอาหารหรือทำกิจกรรมร่วมกัน โดยนำเอาประเด็นจริงของการคำนวณเงิน การจัดการข้อมูล และการยืนยันการชำระเงินมาผนวกรวมกันในระบบเดียว

โปรเจกต์นี้มีจุดเด่นที่สำคัญคือ:

- มี UI ที่ใช้งานง่าย
- มี API ที่จัดการ business logic อย่างชัดเจน
- มี database schema ที่สอดคล้องกับความต้องการจริง
- มีการแยก layer ของงานอย่างเป็นระบบ
- เหมาะสำหรับเป็นผลงานสาธิตหรือ portfolio ฝึกงาน

โดยภาพรวมแล้ว Cleft แสดงให้เห็นว่าผู้พัฒนามีความเข้าใจในหลักการของ full-stack development และสามารถนำความรู้ด้าน database, API, auth, UI, และ system design มาประยุกต์ใช้งานจริงได้